

Indian Institute of Technology, Bhilai

Department of Computer Science and Engineering

CSL351: Computer Networks
Assignment 10 — Socket Programming

Name: Harshita Patidar

ID No: *12340920*

Contents

1	Part 1: DNS-Assisted UDP Video Streaming System	2
1.1	Overview	2
1.2	System Architecture	2
1.3	Custom DNS Protocol	3
1.3.1	Data Structures	3
1.3.2	DNS Records File (<code>records.txt</code>)	3
1.4	Component Implementations	3
1.4.1	DNS Server (<code>dns_server.c</code>)	3
1.4.2	Video Streaming Server (<code>video_server.cpp</code>)	4
1.4.3	Client Application (<code>client.cpp</code>)	4
1.5	Challenges Addressed	4
1.6	Compilation & Execution Instructions	5
1.7	Results & Screenshots	6
2	Part 2: Network Troubleshooting Tools	7
2.1	Q1: RTT Measurement Application (UDP Echo)	7
2.1.1	Overview	7
2.1.2	Core Idea	7
2.1.3	Command-Line Interface	7
2.1.4	Results	8
2.2	Q2: Throughput Measurement (iperf-like Application)	9
2.2.1	Overview	9
2.2.2	Core Idea	9
2.2.3	Results	9
2.3	Q3: Traceroute Implementation	11
2.3.1	Overview	11
2.3.2	Core Idea	11
2.3.3	Implementation Details	11
2.3.4	Compilation & Execution	11
2.3.5	Results	12

1 Part 1: DNS-Assisted UDP Video Streaming System

1.1 Overview

This part implements a complete system where a client first resolves a server's domain name using a custom TCP-based DNS resolver, and then uses the resolved IP address to stream live video over UDP. The system comprises three independently running components:

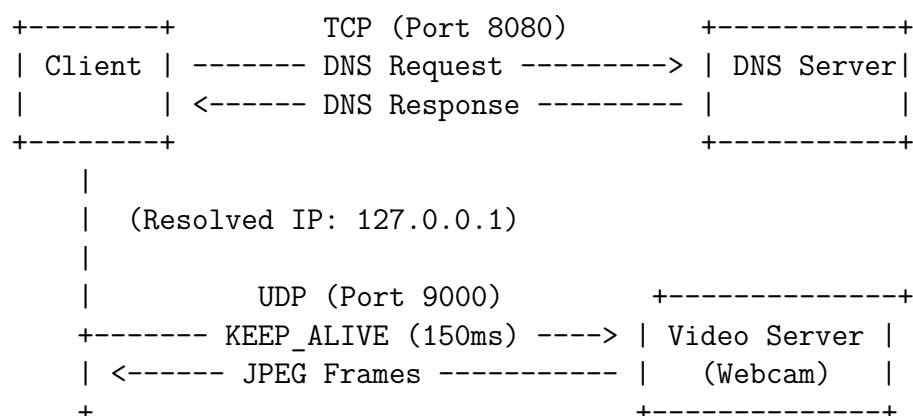
1. **DNS Server (TCP):** Resolves domain names to IP addresses from a local records file.
2. **Video Streaming Server (UDP):** Captures webcam video and streams JPEG-compressed frames over UDP.
3. **Client Application:** Performs DNS resolution (TCP), then receives and displays the video stream (UDP).

1.2 System Architecture

The system follows a two-phase architecture:

- **Phase 1: DNS Resolution (TCP):** The client connects to the DNS server over TCP on port 8080. It sends a custom DNS request containing a **Transaction ID** and a **QNAME** (domain name). The DNS server looks up its **records.txt** file, finds the matching IP address, and returns it as a DNS response with the same Transaction ID.
- **Phase 2: Video Streaming (UDP):** After obtaining the IP address, the client creates a UDP socket and begins sending **KEEP_ALIVE** heartbeat messages every 150ms to the video server on port 9000. The server, upon receiving the first heartbeat, starts capturing video from the webcam, compressing each frame to JPEG format, and transmitting it as a single UDP datagram. The client decodes and displays frames in real-time using OpenCV.

Communication Flow Diagram



1.3 Custom DNS Protocol

1.3.1 Data Structures

The DNS communication uses two custom C structures that are transmitted as raw bytes over the TCP socket:

```
1 // DNS Request Structure
2 typedef struct {
3     int transaction_id; // Unique ID to match request/response
4     char qname[256];    // Domain name to resolve
5 } DNS_Request;
6
7 // DNS Response Structure
8 typedef struct {
9     int transaction_id; // Same ID as the request
10    char ip_address[16]; // Resolved IPv4 address (or "NOT_FOUND")
11 } DNS_Response;
```

Listing 1: DNS Request and Response Structures

1.3.2 DNS Records File (records.txt)

The DNS server maintains a simple text file mapping domain names to IP addresses:

```
1 video.server.com 127.0.0.1
2 test.server.com 192.168.1.10
3 google.com      192.178.211.139
4 iitbhillai.ac.in 192.168.10.115
5 localhost       127.0.0.1
```

Listing 2: records.txt: Domain-to-IP Mappings

1.4 Component Implementations

1.4.1 DNS Server (dns_server.c)

The DNS server is implemented in C and operates as a TCP server. Key design decisions:

- Uses `socket()`, `bind()`, `listen()`, `accept()`, `read()`, and `write()` — all basic socket primitives.
- `SO_REUSEADDR` is set to allow rapid restart during development.
- The `resolve_domain()` function performs a linear search through `records.txt` using `fgets()` and `sscanf()`.
- If a domain is not found, it returns the string "NOT_FOUND" in the IP field.
- Each client connection is handled sequentially (single-threaded), which is sufficient since DNS lookups are fast.

1.4.2 Video Streaming Server (`video_server.cpp`)

The video server captures webcam footage using OpenCV and streams it over UDP. Key implementation details:

- **OpenCV Capture:** Uses `cv::VideoCapture(0)` to open the default webcam with resolution set to 640×480 .
- **JPEG Compression:** Each frame is encoded to JPEG at 50% quality using `cv::imencode()` to reduce bandwidth and ensure frames fit within the UDP maximum datagram size ($\sim 65\text{KB}$).
- **Keep-Alive Mechanism:** The server uses a non-blocking `recvfrom()` (with a 10ms timeout via `SO_RCVTIMEO`) to listen for `KEEP_ALIVE` messages. If no heartbeat is received for more than 200ms, the server stops streaming.
- **Frame Rate Control:** A 30ms sleep between frames yields approximately 30 FPS.
- **Frame Size Check:** Before sending, the server verifies the encoded frame is smaller than 65,500 bytes.

1.4.3 Client Application (`client.cpp`)

The client integrates both phases. Key design decisions:

- **Phase 1 (TCP DNS):** Connects to the DNS server (`127.0.0.1:8080`), sends a request for `video.server.com`, and receives the resolved IP. Uses `write()` and `read()` for TCP communication.
- **Phase 2 (UDP Video):** Creates a UDP socket targeting the resolved IP on port 9000.
- **Keep-Alive Thread:** A separate `std::thread` sends `KEEP_ALIVE` messages every 150ms using `sendto()`. The 150ms interval ensures the server's 200ms timeout is never triggered during normal operation.
- **Frame Display:** Received UDP packets are decoded from JPEG to OpenCV `Mat` using `cv::imdecode()` and displayed with `cv::imshow()`.
- **Graceful Shutdown:** Pressing ESC sets a global flag, which causes the keep-alive thread to stop. The server then detects the heartbeat timeout and stops streaming.

1.5 Challenges Addressed

1. **Packet Loss (UDP Unreliability):** Since UDP does not guarantee delivery, the client simply skips any missed or corrupted frames and continues playback with the next successfully received and decoded frame. This ensures smooth visual continuity.
2. **Frame Serialization & Encoding:** JPEG compression at 50% quality significantly reduces frame sizes from raw BGR ($640 \times 480 \times 3 \approx 921\text{ KB}$) to typically 15–40 KB, well within the UDP datagram limit of $\sim 65\text{ KB}$. This dramatically reduces bandwidth usage.

3. **Real-Time Performance:** A 30ms inter-frame sleep on the server targets ~ 30 FPS. The client uses `cv::waitKey(1)` for minimal display latency. The keep-alive interval (150ms) is tuned to be safely below the 200ms timeout threshold.
4. **Keep-Alive Timer (200ms):** Implemented precisely as specified. The server checks `elapsed > 200` using `std::chrono::steady_clock`, and the client's dedicated thread ensures heartbeats are sent every 150ms, providing a 50ms safety margin.

1.6 Compilation & Execution Instructions

```
1 # Terminal 1: DNS Server
2 cd part1/DNS_Server
3 gcc dns_server.c -o dns_server
4 ./dns_server
5
6 # Terminal 2: Video Server
7 cd part1/Video_Server
8 g++ video_server.cpp -o video_server \
9     'pkg-config --cflags --libs opencv4'
10 ./video_server
11
12 # Terminal 3: Client
13 cd part1/Client
14 g++ client.cpp -o client \
15     'pkg-config --cflags --libs opencv4'
16 ./client
```

Listing 3: Part 1 — Build and Run Commands

1.7 Results & Screenshots

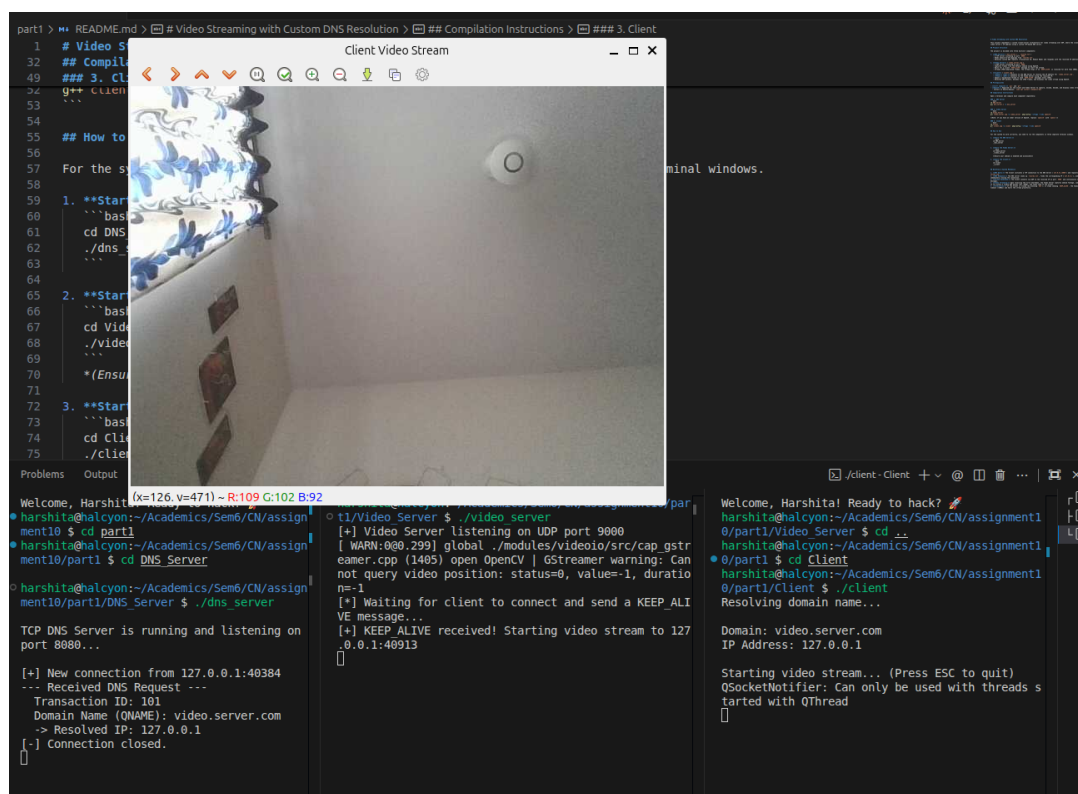


Figure 1: The screenshot above demonstrates the successful execution of the integrated client, confirming the initial TCP-based DNS resolution of the `video.server.com` domain to its local IP address (`127.0.0.1`). Furthermore, it illustrates the seamless transition to the UDP video streaming phase, evidenced by the active OpenCV window effectively displaying the real-time captured frames.

The accompanying terminal validate the backend operations of the streaming system. The DNS Server output confirms the successful reception and resolution of the client's query, correctly mapping the requested domain to its local IP address. Simultaneously, the Video Server terminal logs the receipt of the initial `KEEP_ALIVE` heartbeat, confirming the successful initialization and active transmission of the UDP video stream.

2 Part 2: Network Troubleshooting Tools

2.1 Q1: RTT Measurement Application (UDP Echo)

2.1.1 Overview

This question implements a ping-like RTT (Round Trip Time) measurement tool using UDP echo sockets. The system consists of:

- **Echo Server (`rtt_server.c`):** Receives UDP packets and immediately echoes them back.
- **Echo Client (`rtt_client.c`):** Sends timestamped UDP packets, measures the RTT on receipt of the echo, and reports per-packet timing and overall loss statistics.

2.1.2 Core Idea

The client embeds a sequence number and a `struct timeval` timestamp at the beginning of each packet before sending. The server echoes the entire packet verbatim. When the echo returns, the client extracts the original timestamp from the packet and computes:

$$\text{RTT} = (t_{\text{recv}} - t_{\text{send}})_{\text{sec}} \times 1000 + \frac{(t_{\text{recv}} - t_{\text{send}})_{\text{usec}}}{1000} \quad (\text{in ms})$$

A 1-second socket timeout (`SO_RCVTIMEO`) detects packet loss. If no echo arrives within 1 second, the packet is considered lost.

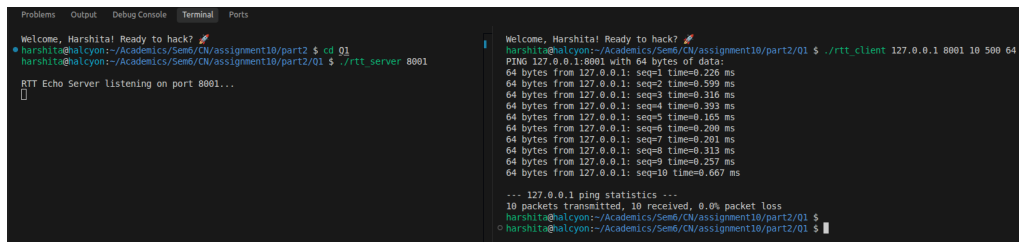
2.1.3 Command-Line Interface

Usage: `./rtt_client <server_ip> <port> <num_messages>`
 `<interval_ms> <packet_size>`

Argument	Type	Description
<code>server_ip</code>	string	IP address of the echo server
<code>port</code>	int	Port number the server listens on
<code>num_messages</code>	int	Number of echo packets to send
<code>interval_ms</code>	int	Interval between consecutive packets (ms)
<code>packet_size</code>	int	Size of each packet in bytes

Table 1: Command-line arguments for the RTT echo client.

2.1.4 Results



```
Problems Output Debug Console Terminal Ports
Welcome, Harshita! Ready to hack?
harshita@halcyon:~/Academics/sem6/CN/assignment10/part2 $ cd Q1
harshita@halcyon:~/Academics/sem6/CN/assignment10/part2/Q1 $ ./rtt_server 8001
RTT Echo Server: Listening on port 8001...

Welcome, Harshita! Ready to hack?
harshita@halcyon:~/Academics/sem6/CN/assignment10/part2/Q1 $ ./rtt_client 127.0.0.1 8001 10 500 64
PING 127.0.0.1:8001 with 64 bytes of data:
64 bytes from 127.0.0.1: seq=1 time=0.226 ms
64 bytes from 127.0.0.1: seq=2 time=0.559 ms
64 bytes from 127.0.0.1: seq=3 time=0.316 ms
64 bytes from 127.0.0.1: seq=4 time=0.393 ms
64 bytes from 127.0.0.1: seq=5 time=0.165 ms
64 bytes from 127.0.0.1: seq=6 time=0.200 ms
64 bytes from 127.0.0.1: seq=7 time=0.201 ms
64 bytes from 127.0.0.1: seq=8 time=0.313 ms
64 bytes from 127.0.0.1: seq=9 time=0.257 ms
64 bytes from 127.0.0.1: seq=10 time=0.667 ms

--- 127.0.0.1 ping statistics ---
10 packets transmitted, 10 received, 0.0% packet loss
harshita@halcyon:~/Academics/sem6/CN/assignment10/part2/Q1 $
```

Figure 2: Q1 — RTT Echo Client and Server output showing per-packet RTT and loss statistics.

2.2 Q2: Throughput Measurement (iperf-like Application)

2.2.1 Overview

This extends Q1 to measure throughput by flooding the echo server with packets (minimal inter-packet delay), computing throughput and average delay in 1-second intervals, and generating time-series plots.

2.2.2 Core Idea

The key changes from Q1 are:

1. **Non-blocking socket:** The socket is set to non-blocking mode (`O_NONBLOCK` via `fcntl()`) so that sending and receiving can be interleaved without the client being stuck waiting for a reply.
2. **Duration-based:** Instead of a fixed number of messages, the test runs for a user-specified duration (in seconds).
3. **Minimal inter-packet delay:** Only a `100μs usleep()` is used to prevent CPU lockup, effectively maximizing packet throughput.
4. **1-second reporting windows:** Every second the client computes:

$$\text{Throughput (Mbps)} = \frac{\text{Bytes Received} \times 8}{10^6}$$

$$\text{Avg Delay (ms)} = \frac{\sum \text{RTT}_i}{\text{Packets Received}}$$

5. **CSV logging:** Metrics are written to `iperf_metrics.csv` for post-processing.
6. **Python plotting:** A separate script generates a dual-axis graph of throughput and delay vs. time.

2.2.3 Results

Time (s)	Throughput (Mbps)	Avg Delay (ms)
1	46.547	0.192
2	46.924	0.191
3	47.841	0.185
4	46.449	0.192
5	47.636	0.187
6	46.850	0.190
7	47.170	0.189
8	46.924	0.190
9	46.531	0.192
10	47.317	0.188

Table 2: Observed throughput and average delay per 1-second interval (localhost test, 1024-byte packets).

The results show a consistent throughput of approximately **47 Mbps** and a very low average delay of approximately **0.19 ms** on the loopback interface. The consistency across the 10-second test demonstrates stable network performance.

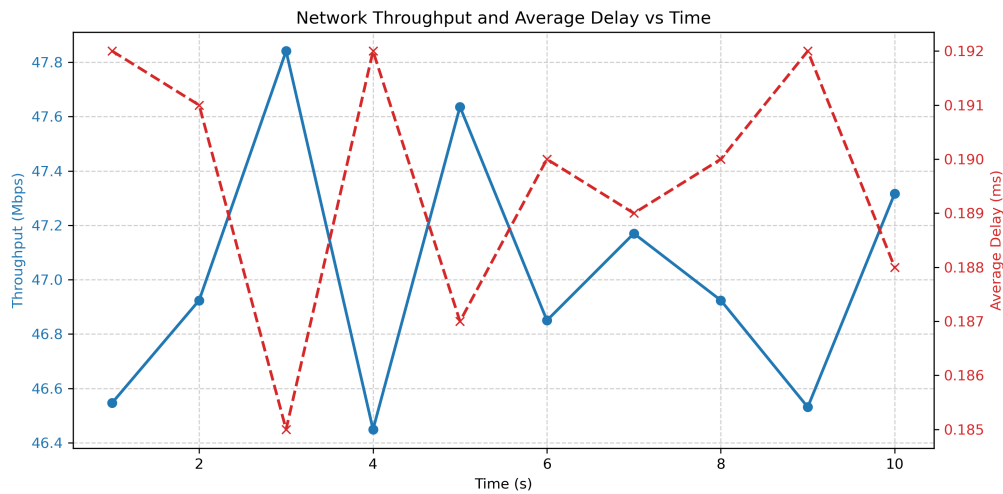


Figure 3: Throughput (Mbps) and Average Delay (ms) vs Time, generated by `plot_throughput.py`.

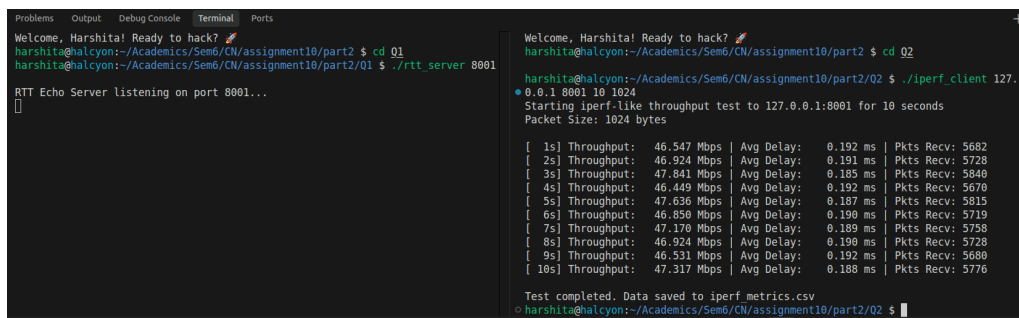


Figure 4: Terminal output of the iperf-like client showing per-second throughput and delay metrics.

2.3 Q3: Traceroute Implementation

2.3.1 Overview

This question implements a custom traceroute tool in C that traces the network path to a destination by sending UDP probe packets with incrementally increasing TTL values and listening for ICMP responses on a raw socket.

2.3.2 Core Idea

Traceroute exploits the IP Time-To-Live (TTL) mechanism:

1. A UDP packet is sent with $TTL = 1$. The first router decrements TTL to 0 and replies with an **ICMP Time Exceeded** message, revealing its IP.
2. TTL is incremented for each hop, revealing successive routers.
3. When the packet reaches the destination, the host responds with an **ICMP Destination Unreachable** (port unreachable), indicating that the trace is complete.
4. For each TTL, **3 probes** are sent to measure RTT variability.

2.3.3 Implementation Details

The implementation uses two sockets:

- **UDP Send Socket (SOCK_DGRAM):** For sending probe packets. The TTL is set per hop using `setsockopt()` with `IP_TTL`. The destination port starts at 33434 (standard traceroute port) and is incremented each hop.
- **Raw ICMP Receive Socket (SOCK_RAW, IPPROTO_ICMP):** For receiving ICMP Time Exceeded and Destination Unreachable messages. A 1-second timeout is set to handle non-responsive hops (marked with *). **Requires sudo privileges.**

Key challenges handled:

- **ICMP parsing:** The received buffer contains an IP header followed by the ICMP header. The IP header length is computed from `ip_hl * 4` to correctly offset to the ICMP header.
- **Noise filtering:** Unrelated ICMP messages (e.g., local loopback noise) are filtered out to display only relevant responses.
- The maximum hop count is set to 30 (`MAX_HOPS`).

2.3.4 Compilation & Execution

```
1 cd part2/Q3
2 gcc traceroute.c -o traceroute
3
4 # Run with sudo (raw socket requires root privileges)
5 sudo ./traceroute google.com
```

Listing 4: Q3 — Build and Run Commands

2.3.5 Results

```

harshita@halcyon:~/Academics/Sem6/CN/assignment10/part2/Q3 $ gcc traceroute.c -o traceroute
harshita@halcyon:~/Academics/Sem6/CN/assignment10/part2/Q3 $ sudo ./traceroute google.com
[sudo] password for harshita-patidar:
traceroute to google.com (192.178.211.100), 30 hops max
 1  10.50.0.1  2.156 ms  2.375 ms  1.848 ms
 2  10.200.10.14  3.408 ms  5.409 ms  8.659 ms
 3  103.147.138.250  76.888 ms  2.877 ms  3.842 ms
 4  117.232.137.122  4.657 ms  4.647 ms  9.132 ms
 5  * * *
 6  * * *
 7  * * *
 8  * * *
 9  * * *
10  * * *
11  142.251.55.120  95.748 ms  102.533 ms  102.198 ms
12  142.251.50.58  205.388 ms  101.885 ms  102.348 ms
13  * * *
14  192.178.254.35  109.273 ms  101.378 ms  102.691 ms
15  209.85.143.251  97.398 ms  103.400 ms  105.663 ms
16  * * *
17  * * *
18  * * *
19  * * *
20  * * *
21  * * *
22  * * *
23  * * *
24  * * *
25  192.178.211.100  186.472 ms  204.338 ms  101.880 ms
harshita@halcyon:~/Academics/Sem6/CN/assignment10/part2/Q3 $ sudo ./traceroute iitbhlai.ac.in
traceroute to iitbhlai.ac.in (192.168.10.115), 30 hops max
 1  10.50.0.1  81.170 ms  101.918 ms  102.229 ms
 2  10.200.10.14  103.124 ms  101.806 ms  102.637 ms
 3  192.168.10.115  10.879 ms  5.962 ms  5.366 ms
harshita@halcyon:~/Academics/Sem6/CN/assignment10/part2/Q3 $

```

Figure 5: Traceroute output showing network hops to a destination with RTT for each probe.